Coverage Path Planning for a Swing-Arm Concrete Troweling Robot in Complex Environments

Yuzhi Pan

Abstract

Concrete floor troweling must cover large areas within a limited construction window. In workspaces partitioned by walls, columns, and openings, a swing-arm robot must plan both base traversal and end-disc coverage. Most existing coverage path planning methods assume a fixed tool footprint and therefore cannot represent the time-varying sweep produced by an actuated arm or generate a continuous, executable route in a complex map. This paper presents a hierarchical coverage path planning method for known two-dimensional environments with obstacles. The method first organizes the workspace using axis-aligned rectangular candidates generated in two scan directions, followed by hole filling and connectivity completion. It then determines a global visitation order and synthesizes coordinated base–arm motions. Coverage is evaluated from the actual area swept by the circular troweling disc along the complete trajectory, while the effects of base speed and arm angular velocity on sweep continuity and execution time are explicitly included. On two test maps, the proposed method achieves full-trajectory coverage ratios of 94.11% and 95.73%, respectively, and both generated routes pass the motion-feasibility checks. By contrast, the connecting segments produced by a fixed-width baseline leave the valid workspace by 4.51 m^2 and 20.00 m^2, making those routes non-executable. Experiments on five maps further show that the bidirectional candidate strategy attains the highest mean full-trajectory coverage ratio (94.32%) and the lowest mean total execution time (2316.60 s). These results support integrating complex-map decomposition, active arm motion, and continuous swept-area evaluation within a single planning pipeline. Validation in dynamic construction environments and on a physical robot remains future work.

Keywords: coverage path planning; construction robotics; concrete troweling; swing-arm robot; area decomposition

1 Introduction

Construction work entails substantial safety and health risks. The U.S. Bureau of Labor Statistics reported 1,032 fatal occupational injuries in construction and extraction occupations in 2024; common events included falls, slips, trips, and transportation incidents involving construction vehicles or machinery [1]. Beyond traumatic injuries, work with silica-containing materials such as concrete and mortar can expose workers to high concentrations of respirable crystalline silica over long periods, increasing the risks of silicosis, lung cancer, and chronic obstructive pulmonary disease [2]. Concrete floor leveling and troweling further combine a large work area, repeated passes, and a narrow time window governed by concrete setting. Manual work therefore requires sustained physical exertion and repetitive motion. Field physiological measurements of concrete slab placing and finishing crews showed that most manual workers experienced high energy expenditure and cardiopulmonary demand, with a greater risk of physical fatigue than equipment operators [3]. Work quality also depends on operator experience, proficiency, and moment-to-moment condition [4, 5]. Releasing workers from hazardous, repetitive, and physically demanding floor operations while improving the consistency of the finished surface is therefore a problem of direct engineering and social value for construction robotics.

Floor-troweling robots provide a practical route toward mechanization. Early field trials showed that a remotely operated troweling robot could perform multiple operations from wood floating to steel troweling. It treated approximately 1,000 m^2 across five sites and achieved an average productivity of about 317 m^2/h

under the reported test conditions [5]. For an order-of-magnitude comparison, the average production rate of manual steel troweling is approximately 13.9 m^2/h [6]; the reported robotic rate is therefore about 22.8 times the rate of one manual worker. Subsequent work introduced electric actuation, laser localization, task planning, and remote monitoring to ride-on power trowels, allowing them to follow specified paths within a predefined floor area [7].

Most existing troweling robots use a disc mounted beneath or in front of the base. Because the tool remains fixed relative to the base, the base path largely determines the tool path, and back-and-forth routes can be planned with a fixed working width [5, 7]. Swing-arm configurations have recently attracted attention because they enlarge the end-disc workspace without requiring a large base. Such a robot can place its circular disc beside the base or into locally constrained regions while the base remains clear of walls, columns, and openings, thereby partially decoupling the traversable base path from the tool trajectory. The arm can also increase the effective coverage width. For a fixed-disc platform, the nominal width of a straight pass is mainly limited by the disc diameter. A swing-arm platform can instead move the disc laterally while the base advances, so that the effective width depends on the arm length, angular range, and disc diameter and can exceed the disc diameter [8]. At the same base speed, provided that the swing frequency keeps consecutive sweeps contiguous without introducing gaps, the theoretical covered area per unit time can increase from approximately $v(2R)$ for a fixed disc to vw_{eff} for a swing arm, where $w_{\mathrm{eff}} > 2R$. Previous coverage-planning research likewise shows that repeated coverage, average velocity, and the smoothness of turns jointly affect task time and energy consumption [9, 10]. The swing-arm configuration may also reduce unnecessary driving and sharp turns on wet concrete and provide more admissible poses near corners, narrow-passage entrances, and obstacles [8].



Figure 1: A swing-arm troweling robot operating on site. The tracked base, swing arm and actuator, and circular end disc are visible. Image source: Shangguan News; photograph provided by China Construction Eighth Engineering Division.

Mechanization and teleoperation, however, do not by themselves provide autonomy. A human teleoperator must continuously observe the floor and machine state while coordinating translation, steering, and troweling. In spaces divided by walls, columns, embedded components, and openings, occluded views and fragmented passages increase the burden of global route selection and can cause backtracking, detours, missed regions, and repeated work. Experiments with teleoperated mobile robots also show that manual driving time increases with environmental difficulty and depends on operator skill and the level of obstacle-avoidance assistance [11]. Existing swing-arm systems commonly rely on human–robot

cooperation or predefined trajectories and do not yet generate complete coverage routes automatically for complex construction maps [8]. Continuous teleoperation also requires a skilled operator to remain on duty, adding training and on-site supervision costs. Empirical studies of construction-robot adoption further identify economic factors, work culture, and uncertain business returns as barriers to deployment [12, 13]. Without autonomous coverage planning, these operational burdens remain even after the troweling process is mechanized.

This task is an instance of coverage path planning (CPP): the tool must sweep the target area while avoiding obstacles and controlling missed coverage, repeated coverage, transfer distance, and execution time [9]. Existing approaches include back-and-forth sweeps after cellular decomposition, grid or spanning-tree traversal, and strip generation with curvature-constrained connections for fixed-width implements. These approaches respectively address polygonal or occupancy-grid maps, regular regions, and applications such as agriculture in which the tool footprint is fixed [14, 15, 16, 17, 18, 19, 20]. Mobile-manipulator methods can plan local motions for a base and an end tool, while SatAOI can identify the workable area of a swing-arm troweling robot in an obstacle map; neither, however, provides a complete continuous coverage route for a complex two-dimensional floor [8, 21]. Section 2 gives a detailed classification and discusses these boundaries.

CPP for a swing-arm troweling robot introduces three interdependent requirements. First, the planner must represent a complex free space containing openings, dense obstacles, and narrow connections. Second, at a fixed base pose, it must distinguish the disc positions and whole-robot collision states associated with different arm angles. Third, while the base moves continuously, actual coverage must be determined from arm angular velocity and the continuous disc sweep rather than from a static reachability envelope. Simply dilating the base path into a constant-width strip cannot satisfy all three requirements. Consequently, prior work lacks a complete planning chain from a construction map through decomposition and visitation order to coordinated base–arm motion and end-tool coverage evaluation.

To close this gap, this paper presents a hierarchical CPP method for known two-dimensional environments with obstacles. Bidirectional rectangular candidates and connectivity completion represent the complex free space. A feasible transition graph then organizes global visitation, and first visits and repeated transit visits are instantiated as local swing-assisted coverage motions and transfer motions, respectively. Finally, the coverage ratio and execution time are computed from the region actually swept by the circular disc along the continuous trajectory. The same robot model is thus used for map representation, arm behavior, and trajectory evaluation.

The remainder of the paper is organized as follows. Section 2 reviews the development of concrete-troweling robots and the challenges posed by obstacle-rich construction environments, compares representative CPP methods, and summarizes the unresolved research gap. Section 3 presents the robot and environment models, rectangular-candidate decomposition, connectivity completion, global visitation, and swing-assisted coverage within each rectangle. Section 4 evaluates the method through a baseline comparison, candidate-scan strategy experiments, and a speed-sensitivity study. Section 5 summarizes the experimentally supported contributions, limitations, and future work.

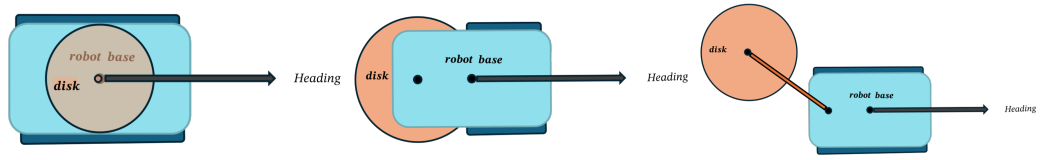
2 Related Work

2.1 Concrete Troweling Robots and Application Challenges

After concrete has been placed and initially leveled, it must be troweled repeatedly before setting and hardening. Troweling removes local height variations and rough texture, redistributes the surface paste, and produces a flat, dense finish through compaction and polishing. The operation covers a large area and is highly repetitive, while its start time, speed, and number of passes are constrained by the state of concrete setting. Starting too early can disturb an unstable surface; starting too late can prevent the desired flatness and finish. Site practice has therefore long depended on the experience and continuous involvement of skilled workers [4, 5]. A troweling robot moves the disc and treats the surface at a relatively

stable speed and cadence, reducing physical workload and the effect of worker-state variability on quality consistency [5, 22]. Its role is thus not merely to replace a hand tool, but to convert a process that requires continuous manual operation into one that a mechanical system can execute consistently.

Existing troweling robots can be grouped into three broad configurations according to the position of the disc relative to the base. In the first, the circular disc is mounted under or near the center of the base. This compact and mechanically stable arrangement is suitable for fixed-width operation on open floors, but the tool footprint essentially translates with the base path. In the second, the disc is mounted in front of the base, making the forward work area easy for an operator or sensor to observe, but requiring the base to approach the treatment boundary and offering limited lateral reach. In the third, the disc is mounted at the end of a swing arm. Controlled relative motion between the base and disc then allows the robot to cover lateral regions, areas near obstacles, and narrow-passage entrances while the base remains at a safe distance. This paper focuses on the third configuration for two reasons. The arm can actively reposition the disc relative to the base, making coverage near walls, columns, openings, and narrow passages more flexible. In addition, at the same base speed and with a contiguous sweep, the effective coverage width can exceed the disc diameter, increasing the area treated per unit time [8, 9, 10]. Both advantages arise from coordinating base translation with arm swinging and cannot be captured by a conventional fixed-width strip model.



(a) Underbody or center-mounted disk (b) Front-mounted disk (c) Swing-arm-mounted disk

Figure 2: Three arrangements of the troweling disc relative to the base: (a) disc beneath or near the center of the base; (b) front-mounted disc; and (c) disc at the end of a swing arm. Arrows indicate the direction of base travel.

The swing-arm arrangement enlarges the platform workspace by mounting the disc at the end of a rotating long arm. Unlike a disc fixed beneath or in front of the base, the arm can treat lateral regions while the base maintains clearance and can create multiple base–tool configurations by changing its angle. This kinematic redundancy aids coverage near walls, columns, and narrow-passage entrances and may reduce base travel and sharp turns on wet concrete. The additional degree of freedom also makes the robot state more complex: x_c and y_c locate the center of the base in the ground frame, θ is the base heading, and ϕ is the arm angle relative to the base. Together, the four variables determine the poses of the base, arm, and disc in the construction plane. A region traversable by the base is not necessarily reachable by the disc, and a disc-reachable pose is not necessarily collision-free for the complete mechanism. Planning must therefore check the geometry of all three components and coordinate the base and arm throughout continuous motion [8]. The capability gained from the mechanical arrangement consequently depends on whether the control system can organize and execute the associated coordinated motion.

The evolution of concrete-troweling equipment can be understood along two axes: the disc–base relationship and the mode of control. Walk-behind and ride-on power trowels first mechanized the operation, but a worker still walked with or rode on the machine. Because the disc remained fixed beneath or in front of the body, the machine path effectively determined the disc path [7]. Remote-control robots subsequently moved the operator outside the work area. Kobayashi et al. fixed front and rear troweling mechanisms to a lightweight mobile chassis and used wireless teleoperation and taught-motion replay to perform successive operations from wood floating to steel troweling. The system reduced worker exertion,

but the troweling mechanisms still moved rigidly with the chassis, and coverage remained sensitive to operator experience, floor friction, and steering behavior [5].

Research then shifted from teleoperation toward autonomous travel in regular areas. Hwangbo et al. retained the fixed twin-rotor troweling mechanism of a ride-on machine, electrified controls such as steering, blade pitch, and acceleration, and added laser localization, reflective landmarks, and a construction-drawing-based task planner. The resulting machine could work autonomously along a prescribed pattern or planned route [7]. This improved control autonomy without changing the rigid coupling between disc and body. More recent swing-arm robots alter that geometry: a tracked base provides locomotion, while a circular disc on an actively rotating arm moves independently relative to the base, partially decoupling traversable base routes from tool trajectories and enlarging the workable region near obstacles [8]. The evolution of troweling robots therefore encompasses both a progression from manual control through wireless teleoperation to partial autonomy and a transition from a body-fixed disc to an actively positioned one. The latter improves flexibility and efficiency in complex environments but introduces a new coordinated base–arm planning problem.

Among existing systems, the swing-arm platform considered by SatAOI is closest to the target of this paper. Mechanically, it retains active relative motion between the end disc and the base and can use multiple base–arm poses near walls, columns, openings, and narrow passages. At the planning level, SatAOI begins with an obstacle grid, explicitly models the base, arm, and disc and their collision rules, and performs a global search to identify mutually connected regions that the robot can enter and in which the disc can operate [8]. It therefore combines the spatial flexibility of a swing arm with environment-model-based autonomous decision making and provides a more complete foundation for automated troweling in complex construction scenes than a fixed tool or a purely teleoperated platform.

SatAOI nevertheless outputs a workable area rather than a complete motion route that covers it. It answers where the robot can work, but not in what order the regions should be visited, how the base should transfer between them, or how the base and arm should coordinate within each region. Key steps—area decomposition, global visitation, local coverage, and speed coordination—are still missing between AOI delimitation and an executable CPP solution. This is the direct starting point of the present study.

2.2 Coverage Path Planning Methods

Existing CPP methods can be compared along two dimensions: map-representation capability and fidelity of robot-behavior modeling. The first describes whether a planner can represent irregular boundaries, multiple obstacles, holes, and narrow connections and use them to establish adjacency and feasible transfers between coverage regions. The second describes whether the planner captures the physical dimensions, relative poses, motion modes, and collision constraints of the robot and tool rather than replacing the operation with a point, disc, or fixed-width strip. These dimensions define four broad capability classes. Idealized coverage methods address regular spaces with simplified robot models and mainly establish fundamental coverage mechanisms. Map-enhanced methods represent complex maps but retain a fixed tool footprint. Platform-constrained methods describe vehicle or manipulator motion constraints but primarily target regular or local workspaces. Environment–platform coupled methods preserve both complex environmental structure and realistic robot behavior.

Swing-arm troweling CPP requires both capabilities of the fourth class. Walls, columns, reserved openings, and narrow corridors divide the floor into irregular regions with complex connectivity, which the planner must represent and visit continuously. Meanwhile, disc position is jointly determined by base motion and active arm motion; coverage and collision status vary continuously with base heading and arm angle and cannot be approximated by a fixed footprint. A complete, continuous, and executable route therefore requires the construction map and the coordinated base–arm–disc behavior to be represented in the same planning model.

The first class comprises classical coverage algorithms that express the core coverage idea clearly but generally close the problem only for idealized maps and simplified robot models. Representative methods include Boustrophedon Cellular Decomposition (BCD) and Spanning Tree Coverage (STC).

BCD partitions an area where scan-line connectivity changes and generates back-and-forth sweeps within each cell [14]; STC builds a spanning tree over a free-space grid and systematically traverses its boundary [15]. These methods establish the respective roles of cellular decomposition and graph traversal, but their coverage primitive is typically a point robot, disc robot, or fixed-width tool. Applied directly to swing-arm troweling, they cannot distinguish the relative poses of the base, arm, and disc or represent how arm-angle changes affect the instantaneous footprint and collision risk.

The second class offers stronger complex-map representation and includes reproducible open-source frameworks, but still models the robot and tool mainly through a fixed footprint. Examples include ETH Zurich's `polygon_coverage_planning`, Fields2Cover for agricultural vehicles, and the Nobleo Full Coverage Path Planner for ROS occupancy grids. The first handles general polygons with holes, uses BCD or trapezoidal decomposition to generate cells, constructs multiple sweep candidates for each cell, and performs global selection and collision-free transfer through an Equality Generalized Traveling Salesman Problem and a visibility graph [16, 17]. Fields2Cover supports non-convex fields, obstacles, headlands, swaths, route ordering, and curvature-constrained Dubins or Reeds–Shepp connections [18, 19]. Nobleo's planner implements a Backtracking Spiral Algorithm/Spiral-STC-type method on a ROS occupancy grid and permits separate robot and tool radii [20]. These frameworks show that complex boundaries, obstacles, and engineering interfaces can be addressed systematically. Their tool width, implement width, or tool disc nevertheless remains essentially fixed relative to the base and cannot represent the time-varying sweep caused by continuous changes in ϕ.

The third class emphasizes robot motion constraints, reachability, and collision relationships, but usually does not target complete coverage of a complex two-dimensional floor. It includes the nonholonomic vehicle connections in Fields2Cover and mobile-manipulator surface-coverage methods for tasks such as spray disinfection. Fields2Cover incorporates minimum turning radius and vehicle kinematics [18, 19], but targets agricultural swaths with a fixed implement width. Mobile-manipulator studies explicitly handle the base, arm, end tool, reachability, and occlusion. For example, spray-based surface disinfection can generate end-effector paths from three-dimensional surfaces and jointly select base positions and arm motions [21]. Such methods describe how a robot executes a task in greater detail, but generally target regular sites, agricultural strips, or three-dimensional object surfaces rather than jointly performing obstacle handling, area decomposition, global traversal, and continuous ground-coverage evaluation on a known two-dimensional construction map.

Research has thus advanced in both complex-map representation and robot-behavior modeling, but the two have not yet been unified for swing-arm floor coverage. CPP frameworks with strong map support usually assume a fixed tool footprint and cannot describe the dynamic coverage created by a rotating arm. Methods that model vehicle kinematics, manipulator reachability, and component collisions in detail generally focus on regular sites, local operations, or three-dimensional surfaces and do not fully decompose and continuously traverse a complex two-dimensional construction map. SatAOI considers both an obstacle map and the geometry of the base, arm, and disc, but stops at delimiting the workable area and does not produce a visitation order, transfer route, or coordinated coverage trajectory within each region [8]. The central gap is therefore a swing-arm troweling CPP method that combines complex-map processing with high-fidelity robot-behavior modeling and generates a complete, continuous, executable route directly from the construction map.

Table 1 compares representative approaches along these two dimensions. Work spanning more than one class is placed according to its principal contribution: Fields2Cover is treated as map-enhanced and SatAOI as platform-constrained. The stated limitations concern direct application to a mobile base with an active arm and circular disc; they are not general judgments of the methods or software.

Table 1: Representative CPP methods and their applicability to swing-arm troweling.

Category	Representative method	Map and tool model	Relevance to swing-arm troweling
Idealized coverage	BCD [14]	Cellular decomposition; fixed footprint	Organizes back-and-forth sweeps within cells, but does not model an active arm or its continuous sweep.
Idealized coverage	STC [15]	Grid spanning tree; simplified robot	Systematically traverses a grid, but cannot exploit the time-varying tool position created by arm motion.
Map-enhanced	Polygon CPP [16, 17]	Polygon with holes; fixed tool	Provides alternative sweeps and obstacle-avoiding transfers, but does not represent the arm's instantaneous footprint.
Map-enhanced	Fields2Cover [18, 19]	Non-convex field; fixed-width implement	Includes swaths, route ordering, and curvature-constrained connections; the fixed implement model omits arm motion.
Map-enhanced	Nobleo FCPP [20]	Occupancy grid; fixed tool disc	Integrates grid coverage with a navigation stack, but does not coordinate arm angle, arm speed, and base speed.
Platform-constrained	Mobile-manipulator surface coverage [21]	Three-dimensional surface; mobile manipulator	Jointly plans base placement and end-tool motion, but does not completely traverse a complex two-dimensional floor.
Platform-constrained	SatAOI [8]	Construction obstacle map; base, arm, and disc	Determines workable regions and connectivity, but does not generate a visitation order or continuous coverage trajectory.
Proposed method	Hierarchical swing-arm CPP	Two-dimensional obstacle map; time-varying disc sweep	Links rectangular decomposition, visitation, local swinging, and trajectory evaluation; currently assumes a static map and uses heuristic search.

2.3 Summary and Research Gap

Prior work provides several capabilities needed here, but has not solved complex-map coverage within one unified task. BCD, STC, and related frameworks can represent two-dimensional free spaces containing holes, non-convex boundaries, or multiple obstacles and can decompose regions, generate swaths, or traverse grids. With a tool fixed to the base, they can arrange adjacent sweeps according to a prescribed width. Fields2Cover additionally generates turns and connections for vehicles whose direction and minimum turning radius prevent arbitrary lateral motion. Mobile-manipulator research addresses end-tool reachability and collision relationships, while SatAOI uses the geometry among the base, arm, disc, and obstacles to delimit regions the robot can enter and the disc can treat [8, 14, 15, 16, 17, 18, 19, 20, 21]. These abilities remain distributed across different task models and applications: complex-map CPP usually assumes a fixed footprint, detailed robot models are commonly used for local or regular workspaces, and SatAOI terminates at workable-area delimitation. For the specific objective of generating a complete executable route for a swing-arm troweling robot in a known, complex, two-dimensional construction map, three coupled gaps remain.

First, existing CPP methods do not model active arm behavior sufficiently to plan the actual coverage of the end disc. In conventional CPP, the tool is fixed relative to the base, so determining the base route also determines tool position and coverage. For a swing-arm robot, the same base position and heading admit multiple arm poses and therefore different disc positions, swept regions, and whole-robot collision states. Approximating this behavior by the maximum arm reach or an enlarged fixed width confuses geometric reachability in one pose with coverage throughout continuous motion and cannot test the combined collision state of the base, arm, and disc during execution. SatAOI models the geometry and collision relationships among these components but uses them primarily to determine workable positions rather than continuous coverage motion [8]. The present method therefore preserves the relationship among base motion, arm rotation, and the circular-disc sweep during both trajectory generation and evaluation.

Second, complex-map decomposition and local swing motion have not been integrated into a computationally practical planning chain. Existing complex-map CPP frameworks handle polygons with holes, non-convex boundaries, or occupancy grids and produce partitions and global routes, but their local coverage actions typically rely on a fixed tool footprint [16, 17, 18, 19, 20]. Mobile-manipulator methods plan local base and end-tool motions in detail but generally do not decompose and continuously traverse an entire complex two-dimensional floor [21]. SatAOI identifies workable regions and their connectivity but does not proceed to visitation ordering, inter-region transfers, or coordinated intra-region coverage [8]. Directly enumerating decompositions, visitation orders, and arm trajectories together would cause the number of combinations to grow with the obstacles and work units, undermining computational efficiency and field practicality. The proposed method instead organizes complex-space decomposition, region connectivity, global visitation, and coordinated base–arm coverage as a hierarchical heuristic pipeline that converts map-level decisions into continuous work routes while controlling the candidate-search space.

Third, a static reachability envelope does not guarantee complete coverage during dynamic execution, and prior work does not explicitly model the matching between base linear velocity and arm angular velocity. Maximum arm reach indicates only which locations the disc can touch in some poses. During continuous base advance and periodic arm swinging, coverage continuity depends on the number of sweeps per unit distance. If the base is too fast or the arm too slow, the longitudinal spacing between consecutive sweeps increases, leaving untreated gaps even when the target lies inside the geometric reach envelope. Evaluation based only on that envelope or on an untimed geometric path therefore mistakes "reachable" for "covered." This paper incorporates the relationship between base and arm velocities into the coverage analysis and calculates coverage ratio and task time from the area actually swept during continuous motion, exposing the joint effect of speed parameters on gaps and time cost.

In summary, the core research gap is the absence of an end-to-end CPP method that, in a known two-dimensional construction environment with dense obstacles, connects active-arm behavior modeling, global–local coverage planning for a complex map, and continuous swept-area evaluation under coupled base and arm velocities. This work aims to complete the planning chain from a construction map and swing-arm robot model to a complete, continuous, executable coverage trajectory, with final coverage quality determined by the disc sweep under the specified velocity relationship.

3 Methodology

This study addresses the planning problem that maps a known, static, two-dimensional obstacle environment to a complete coverage trajectory for a swing-arm troweling robot. Walls, columns, and openings divide a construction floor into a free space with complex geometry and connectivity, while the disc position and footprint are jointly determined by the base pose and arm angle. The planner must therefore organize global visitation of the free space and convert coverage of each region into continuous, coordinated base–arm motion. Subject to obstacle-avoidance, geometric, and motion constraints, the objectives are to increase effective coverage of the target floor and reduce both trajectory execution time and planning time. The number of rectangles, their overlap, and the number of inter-rectangle transfers are intermediate measures only; they are converted into proxy costs for polynomial-time heuristics at the respective planning levels.

3.1 Overall Planning Pipeline

The proposed hierarchy follows four stages: region representation, graph visitation, local execution, and coverage evaluation. Figure 3 summarizes the data flow. The inputs comprise a calibrated binary work map M with physical pixel size s_p, an initial robot state q_0, geometric and motion parameters Θ_R for the base–arm–disc mechanism, and planning configuration Ψ_P. These quantities are defined formally in Section 3.2; the present subsection explains how they are progressively transformed into an executable trajectory and coverage measures.

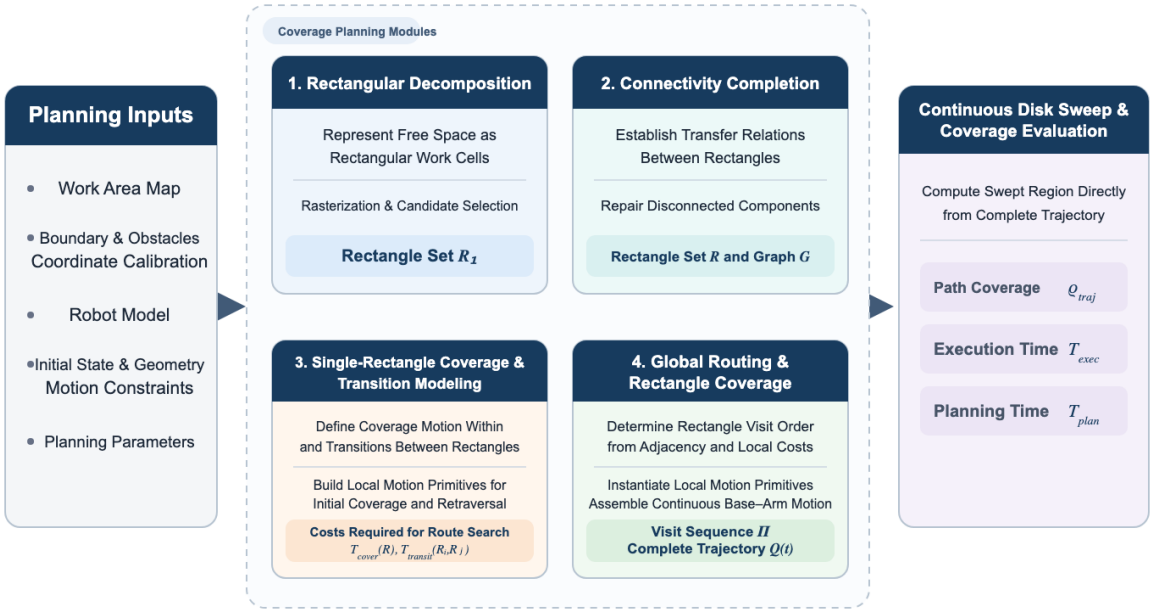


Figure 3: Coverage-planning framework and principal data flow. The inputs on the left are the work map, coordinate calibration, robot model, initial state, and planning parameters. The four central modules successively produce an initial rectangle set $\mathcal{R}_1$, a final rectangle set $\mathcal{R}$ with adjacency graph G, entry- and exit-constrained local motion primitives with time costs, and a visitation sequence Π with complete trajectory $q(t)$. Continuous disc sweeping then yields coverage ratios, execution time, and planning time.

The rectangular-decomposition module first generates candidate rectangles on a conservative planning grid and selects suitable work units according to newly covered area, rectangle count, and overlap, producing the initial set $\mathcal{R}_1$. The connectivity-completion module creates geometric adjacency candidates, removes relationships that fail a base–arm transfer check, and inserts bridging rectangles when needed. Its output is a final rectangle set $\mathcal{R}$ and feasible adjacency graph G. An edge in G therefore means not only that two rectangles touch geometrically, but also that at least one local transfer satisfies the geometric and velocity constraints in Eqs. (3a) and (3b).

For every rectangle, the local-modeling module constructs two primitive types. A first visit uses a coverage primitive, whereas a repeated visit uses a transit primitive. Their durations depend on the entry and exit states and are denoted by $T_{\text{cover}}(R; q^{\text{in}}, q^{\text{out}})$ and $T_{\text{transit}}(R; q^{\text{in}}, q^{\text{out}})$, respectively. These costs, together with the transfer costs on graph edges, enter the global route search. After the visitation sequence Π has been selected, the same primitives are instantiated as first-visit or repeated-visit motions and concatenated into the complete trajectory. The local motion model thus supports both visitation ordering and final trajectory generation.

Starting from q_0, the global module generates a sequence on G that permits repeated transit visits and connects adjacent primitive endpoints, yielding a continuous trajectory $q(t)$ with timing and work-state labels. The evaluation module recovers the disc-center path from $q(t)$ and computes effective-work coverage ρ_{work}, full-trajectory coverage ρ_{traj}, and execution time T_{exec}. The wall-clock time from conservative-grid generation through trajectory evaluation is T_{plan}. The following subsections define the

inputs, states, and metrics, then describe each module and the continuous swept-area evaluation.

3.2 Problem Formulation and Notation

3.2.1 From a Construction Scene to Planning Inputs

The required scene information is first extracted from a CAD/BIM model, site-survey data, or localization and mapping output. For a floor to be troweled, a world-coordinate plane $\mathcal{F}_W$ coincident with the work surface is selected, and the boundary of the current task is specified in that plane. Walls, columns, equipment foundations, embedded components, reserved openings, and areas outside the work boundary are marked as unavailable. The remaining floor inside the boundary on which the disc may operate becomes the target work region. To account for localization error, map error, or additional clearance, unavailable-region contours can be dilated by a margin δ_{map} before rasterization. The classified plane is then rasterized and geometrically calibrated to produce a physically scaled binary work map, which is the direct map input to CPP.

Discretization converts heterogeneous geometry from CAD/BIM, field measurement, and mapping into spatial cells at a common resolution. This gives rectangle generation, adjacency testing, collision checking, and coverage calculation a consistent computational basis. At a resolution of s_p m/pixel, the plane becomes a binary map M: white pixels are obstacle-free target pixels included in the coverage task, whereas black pixels represent walls, columns, openings, out-of-bound areas, and areas excluded by the safety margin. In result and evaluation images, blue pixels denote portions of the target region already swept by the disc. Blue is an overlay on M, not a third input-map class. A rigid transform T_{WM} calibrates image coordinates to world coordinates, so every pixel corresponds to a physical location in metres.

The conversion from planar geometry to the binary map follows uniform labeling and conservative discretization. Regardless of boundary shape, all workable and unavailable regions are projected onto the same plane and classified at one resolution as free or occupied cells. Cells on an obstacle boundary or with ambiguous membership are treated as unavailable to avoid misclassifying obstacle space as workable floor. Straight walls and rectangular foundations map directly to contiguous black cells. Circular columns, curved boundaries, and openings are approximated by staircase boundaries at the selected resolution, optionally enlarged by the safety margin. In regions with irregular boundaries or dense obstacles, the occupancy map preserves the overall shape and connectivity; subsequent rectangle generation, hole filling, and connectivity completion handle them uniformly. The conversion therefore does not require a separate decomposition rule for every geometric shape. Its approximation accuracy is governed jointly by map resolution and safety margin.

After the map has been abstracted, equipment drawings, calibration tests, and controller limits provide the robot model and initial state. The scene input for one planning task is divided into map and robot components:

$$\mathcal{I} = \left\langle \mathcal{I}_{\text{map}}, \mathcal{I}_{\text{robot}} \right\rangle,$$

where

$$\mathcal{I}_{\text{map}} = \left\langle M, s_p, T_{WM} \right\rangle.$$

Here M is the $H_p \times W_p$ binary work map generated from CAD/BIM, survey, or mapping data; s_p converts pixel distance and area to metres and square metres; and T_{WM} is the planar rigid transform from the map frame to the world frame, used to convert planned map coordinates into executable site coordinates.

The robot component is

$$\mathcal{I}_{\text{robot}} = \langle q_0, \Theta_R \rangle.$$

The initial base position, heading, and arm angle in q_0 are obtained from the site-localization system and arm encoder. The parameter set Θ_R contains the base length l_c and width w_c, arm-link collision radius r_a, troweling-disc radius r_d (diameter $2r_d$), arm length L, distance d_p from the base center to the arm pivot, arm-angle limits, maximum base linear and angular velocities, and maximum arm angular velocity.

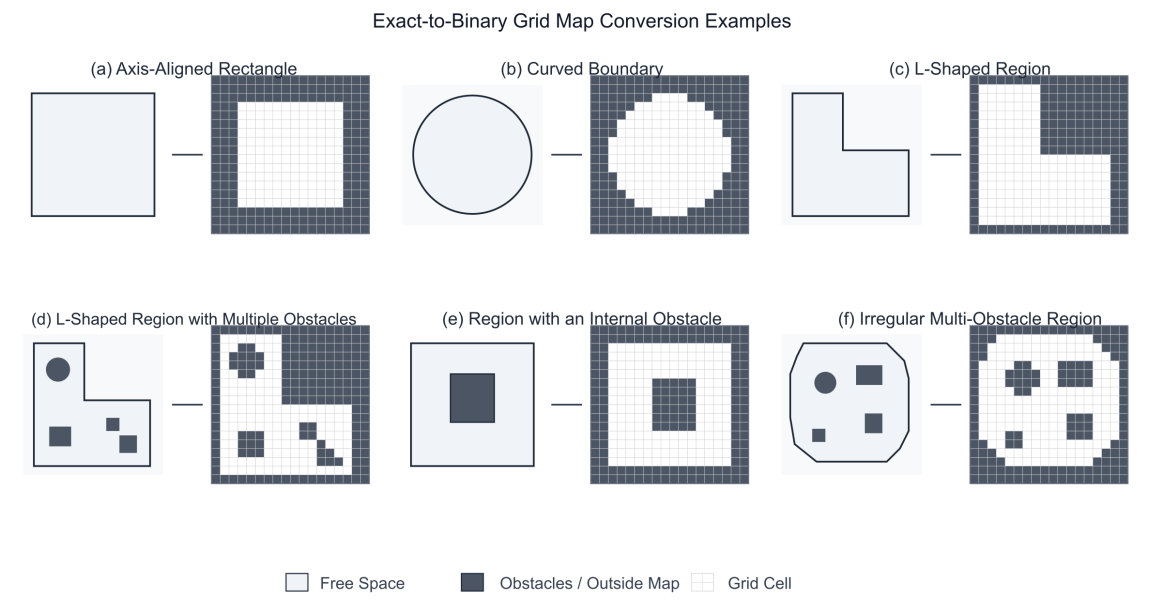


Figure 4: Conversion from exact geometric descriptions to the binary raster input. The left column shows exact two-dimensional work regions and rectangular subregions abstracted from CAD/BIM, survey, or mapping data; the right column shows the binary map M used by the planner. White denotes free target area, black denotes obstacles, openings, or out-of-bound regions, and dashed rectangles indicate approximate cells that may support later rectangular candidates. Examples include rectangular rooms, circular or curved boundaries, L-shaped areas, T-junction or corridor connections, regions with columns or openings, and irregular areas with dense obstacles.

Together, these parameters determine occupied geometry, the disc coverage footprint, and trajectory executability.

In addition to these scene inputs, the algorithm uses a planning configuration Ψ_P: planning-cell size g, minimum rectangle side length, candidate-selection parameters α and β, candidate truncation count, scan and connectivity strategies, and random seed. The configuration controls decomposition and search but is not part of the map or robot model abstracted from the physical scene.

3.2.2 Planning Model and Notation

Let the input map be

$$M \in \{0, 255\}^{H_P \times W_P},$$

where $M_{uv} = 0$ is an obstacle pixel and $M_{uv} = 255$ a free pixel. With physical pixel size s_p, the free-pixel set is

$$\mathcal{P}_F = \{(u, v) \mid M_{uv} = 255\}.$$

All subsequent positions and distances are transformed by T_{WM} into a metric world frame; for clarity, the transform is omitted from the remaining notation.

The robot state is

$$q(t) = \big(x_c(t), y_c(t), \theta(t), \phi(t)\big),$$

with state space $\mathcal{Q} = \mathbb{R}^2 \times S^1 \times [\phi_{\min}, \phi_{\max}]$. Here (x_c, y_c) is the base center, θ the base heading, and ϕ the arm angle relative to the base. Let $\mathbf{p}_c = (x_c, y_c)$ and $\mathbf{e}(\gamma) = (\cos\gamma, \sin\gamma)$. For arm length L and base-center-to-pivot offset d_p, the center of the circular disc is

$$\mathbf{p}_d(q) = \mathbf{p}_c - d_p\mathbf{e}(\theta) + L\mathbf{e}(\theta + \pi + \phi). \tag{1}$$

Equation (1) is the sum of two translations: from the base center to the arm pivot and from the pivot to the disc center. As shown in Fig. 5, $\mathbf{e}(\theta)$ points along the base heading, and the pivot lies d_p behind the base

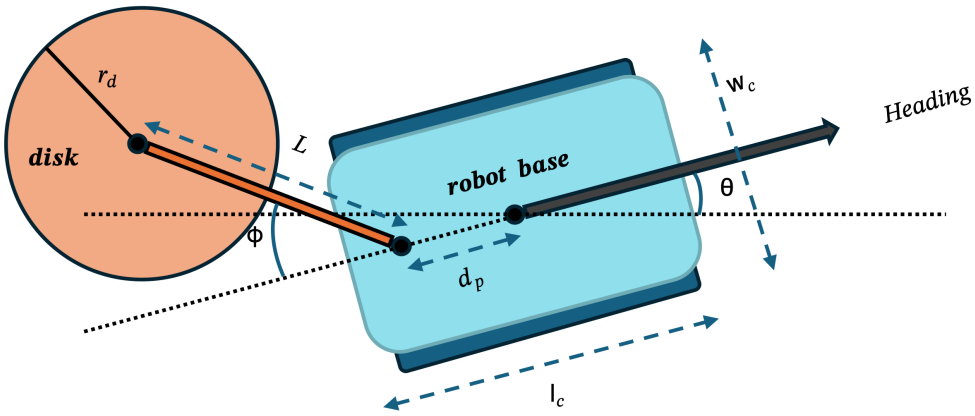


Figure 5: Geometry and motion parameters of the base–arm–disc system. The diagram identifies the base center, heading θ, relative arm angle ϕ, disc radius r_d, arm length L, base-center-to-pivot offset d_p, and base length and width l_c, w_c. These quantities determine the robot's occupied geometry and the disc-center position in Eq. (1).

center, so

$$\mathbf{p}_p = \mathbf{p}_c - d_p \mathbf{e}(\theta).$$

The arm angle ϕ is defined as the signed rotation from the rearward base axis, positive counterclockwise. Since the rearward axis is rotated by π from the forward heading, the pivot-to-disc direction is $\theta + \pi + \phi$, and its displacement is $L\mathbf{e}(\theta + \pi + \phi)$. For example, when $\theta = 0$ and $\phi = 0$, the base faces the positive world x-axis, the pivot is at $(x_c - d_p, y_c)$, and the disc center is at $(x_c - d_p - L, y_c)$: the arm extends directly behind the base. Changing θ rotates the pivot and arm with the base; changing ϕ at fixed θ moves the disc center along a radius-L arc about the pivot.

For disc radius r_d, the instantaneous work region at one pose is

$$\mathcal{D}(q) = \{\mathbf{z} \in \mathbb{R}^2 \mid \|\mathbf{z} - \mathbf{p}_d(q)\|_2 \le r_d\}. \tag{2}$$

The region swept by a continuous trajectory $q : [0, T] \to \mathcal{Q}$ is

$$C(q) = \bigcup_{t \in [0,T]} \mathcal{D}(q(t)). \tag{3}$$

To distinguish disc reachability from whole-robot feasibility, let $\mathcal{O}_M$ be the unavailable region corresponding to black pixels in world coordinates, $\mathcal{B}(q)$ the rotated $l_c \times w_c$ base rectangle, and $\mathcal{A}(q)$ a capsule of radius r_a around the arm link. A state is geometrically feasible only if

$$\big(\mathcal{B}(q) \cup \mathcal{A}(q) \cup \mathcal{D}(q)\big) \cap \mathcal{O}_M = \varnothing. \tag{3a}$$

All feasible states form $\mathcal{Q}_F \subseteq \mathcal{Q}$. For a motion segment $q_s : [0, T_s] \to \mathcal{Q}$, the method requires $q_s(t) \in \mathcal{Q}_F$ for every $t \in [0, T_s]$ and

$$|v(t)| \le v_{\max}, \qquad |\dot{\theta}(t)| \le \omega_{\max}, \qquad |\dot{\phi}(t)| \le \omega_{a,\max}. \tag{3b}$$

Straight motion, in-place rotation, arm adjustment, and other primitives are adaptively subdivided, and Eq. (3a) is checked over every subinterval. Consequently, later rectangle adjacencies and entry states are candidates only; a transfer enters the final planning graph only after passing this check.

Under Eqs. (3a) and (3b), the algorithm is assessed at three levels: work quality, trajectory-execution efficiency, and planning efficiency. The corresponding objectives are

$$\max \rho_{\text{work}}(q), \qquad \min T_{\text{exec}}(q), \qquad \min T_{\text{plan}}. \tag{4}$$

The effective-work coverage ratio $\rho_{\text{work}}(q)$ is the fraction of the target area swept by the disc during work-labeled segments. The nominal execution time $T_{\text{exec}}(q)$ runs from the initial state to completion and includes the initial connection, effective work, turns, arm adjustments, and inter-region transfers. The wall-clock planning time T_{plan} covers the algorithm from conservative-grid generation through trajectory assembly and coverage evaluation.

These measures play distinct roles. The task-level quality measure ρ_{work} is encouraged indirectly through free-space rectangle representation, rewards for new coverage, and overlap penalties, then computed from the continuous disc sweep in Eqs. (1)–(3) after the complete trajectory is available. The route-level objective T_{exec} supplies time costs to local primitives and global visitation candidates; the algorithm selects feasible trajectories with low proxy cost but does not claim a global minimum. The planning-time measure T_{plan} is not a decision variable of q; it is a design objective controlled by hierarchical decomposition, polynomial-time heuristics, and candidate truncation. This separation motivates the hierarchical heuristic framework.

3.3 Conservative Rasterization and Rectangular Decomposition

The decomposition module first converts the pixel map into a coarser planning grid. Let each square planning cell contain $g \times g$ pixels, and let Ω_{ij} be the pixel block associated with planning cell (i, j). Conservative rasterization gives

$$B_{ij} = \begin{cases} 1, & \text{if } M_{uv} = 255, \quad \forall (u, v) \in \Omega_{ij}, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Thus, $B_{ij} = 1$ means that the planning cell can support rectangle generation; $B_{ij} = 0$ means that it contains or touches an obstacle. The rule sacrifices some free area near obstacles but guarantees that subsequent rectangular candidates do not cross known obstacles.

On B, the algorithm scans maximal contiguous runs of free cells by rows and by columns, extending them across adjacent scan lines as long as every enclosed cell remains free. This produces axis-aligned candidates of the form

$$R = [x_1, x_2] \times [y_1, y_2], \tag{6}$$

with all planning cells inside R free. Candidates whose width or height is below a specified minimum are removed. The remainder are ordered first by decreasing area and, for equal areas, by increasing row and column coordinates of the upper-left and then lower-right corner. Only the first $C_{\max}$ candidates from each scan direction are retained. This deterministic truncation controls rapid growth in candidate count while making the candidate set repeatable for fixed inputs and parameters.

Candidate selection is formulated as a penalized maximum-coverage problem. Let $\mathcal{U}(R)$ be the set of planning cells covered by R and $\mathcal{A}$ the cells already covered by selected rectangles. The marginal utility of selecting R is

$$\Delta(R \mid \mathcal{A}) = |\mathcal{U}(R) \setminus \mathcal{A}| - \alpha - \beta |\mathcal{U}(R) \cap \mathcal{A}|. \tag{7}$$

Cardinalities are measured in planning cells, so α has units of cells and β is dimensionless. The first term rewards new coverage; the other terms penalize rectangle count and overlap. The algorithm repeatedly selects the row-scan candidate with the largest positive marginal utility, using the deterministic ordering above to break ties. It then processes the column-scan pool relative to the current $\mathcal{A}$, so column-oriented rectangles mainly supplement vertical structures poorly represented by row scans. The two greedy stages produce the initial selection.

Candidate truncation or the penalties may leave a small number of free cells uncovered. A deterministic hole-filling stage therefore selects the first uncovered free cell in row-major order as a seed, enumerates maximal axis-aligned obstacle-free rectangles containing the seed, and selects the largest. Remaining ties are resolved by rectangle width and upper-left coordinates. The selected rectangle is added and $\mathcal{A}$ updated until every usable planning cell belongs to at least one rectangle. The resulting set, before connectivity repair, is $\mathcal{R}_1$. Minimum side length, $C_{\max}$, α, β, and scan direction are included in Ψ_P.

3.4 Connectivity Completion and Final Segmentation

Covering every free planning cell with rectangles does not ensure that the robot can move continuously between them. The method therefore constructs an undirected graph

$$G = (V, E), \tag{8}$$

over $\mathcal{R}_1$, with node $v_i \in V$ representing rectangle R_i. Shared boundaries, contact, and overlap generate geometric adjacency candidates only. For each candidate pair (R_i, R_j), a finite set of handoff poses is generated in the intersection or along the closest opposing boundaries. The algorithm then attempts to connect an exit state of the first rectangle to an entry state of the second. Edge (v_i, v_j) is added only if at least one transfer primitive satisfies Eqs. (3a) and (3b). The edge stores the minimum time proxy c_{ij} among all feasible handoffs. Graph G consequently represents executable robot transfers, not merely geometric contact between rectangles.

If G has multiple connected components, the algorithm searches the unselected candidates for bridging rectangles. A candidate R_b is a bridge if, after insertion, it forms feasible edges with rectangles in at least two previously distinct components. Preference is given to rectangles with little new area and low feasible-transfer cost:

$$R_b^\star = \arg\min_{R_b \in \mathcal{B}} \left(|\mathcal{U}(R_b) \setminus \mathcal{A}| + \lambda_b \sum c_{ib} \right), \tag{9}$$

where $\mathcal{B}$ is the current bridge-candidate set, the sum contains only edges used to connect components, and λ_b weights transfer time. Relevant edges and connected components are recomputed after every insertion, until G becomes connected. If $\mathcal{B}$ is empty while G remains disconnected, no single continuous coverage trajectory is supported by the current candidate set and parameters. The method reports connectivity failure and does not proceed to global route planning.

Algorithm 1 summarizes the complete process from conservative rasterization to the final graph. The overlap penalty in Eq. (7) suppresses redundant rectangles within a candidate pool; hole filling represents every free planning cell; and bridging adds rectangles only when the current set cannot form a connected feasible graph. "Connected" always refers to graph connectivity after complete robot-transfer checks, not merely topological connectivity of free cells or the geometric rectangle union.

Let the planning grid be $H \times W$, the truncated candidate count C, the final rectangle count n, and the mean candidate area A cells. Candidate generation has worst-case complexity $O(HW(H + W))$. With a naive set implementation, greedy selection is approximately $O(CnA)$. Adjacency-candidate enumeration is $O(n^2)$, with a finite number of local-motion checks per edge. The minimum side length, candidate truncation, and deterministic ordering jointly limit practical problem size. Since later graph search and local-primitive instantiation grow with n, controlling the rectangle count during decomposition directly affects T_{plan}.

3.5 Single-Rectangle Coverage Primitive

The rectangle-coverage module supplies the local motion primitive. Given a rectangle R, feasible entry pose q^{in}, feasible exit pose q^{out}, and robot parameters, it generates a continuous motion segment. On the first occurrence of R in the global sequence, the robot executes the back-and-forth coverage shown in Fig. 6; if R has already been covered, only an entry-to-exit transit motion is generated. Both primitive types retain the complete robot state, duration, and work-state label.

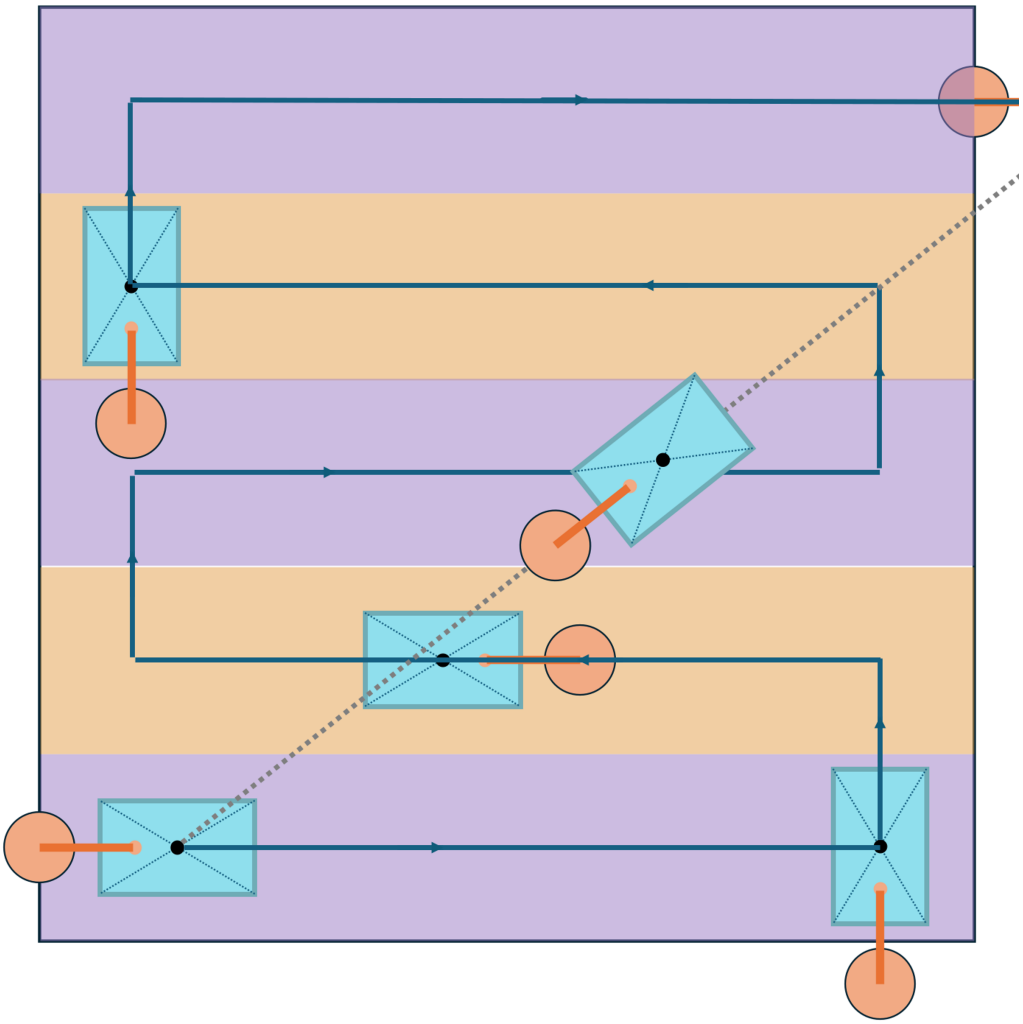


Figure 6: Single-rectangle coverage primitive. The disc entry and exit are determined by neighboring rectangles or the initial state. The local planner compares time proxies for both sweep directions and feasible starting corners, then covers the rectangle through forward base motion and reciprocal arm swinging.

Algorithm 1 Rectangle segmentation and connectivity completion

Require: Conservative free-cell grid B, planning parameters Ψ_P, robot model Θ_R
Ensure: Rectangle set $\mathcal{R}$ and feasible graph G, or connectivity failure

$F \leftarrow \{u : B(u) = 1\}$; $\mathcal{R} \leftarrow \varnothing$; $\mathcal{A} \leftarrow \varnothing$
$(C_r, C_c) \leftarrow \text{ScanAndTruncate}(B, \Psi_P)$
$C_{\text{all}} \leftarrow C_r \cup C_c$
for $C \in (C_r, C_c)$, in this order **do**
  while $\exists R \in C : \Delta(R \mid \mathcal{A}) > 0$ **do**
    $R^\star \leftarrow \arg\max_{R \in C} \Delta(R \mid \mathcal{A})$ ▷ Eq. (7); deterministic ties
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{R^\star\}$; $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{U}(R^\star)$
    $C \leftarrow C \setminus \{R^\star\}$
  end while
end for
while $\mathcal{A} \neq F$ **do**
  $u \leftarrow \text{FirstUncovered}(F \setminus \mathcal{A})$
  $R^\star \leftarrow \text{LargestMaximalRectangle}(B, u)$ ▷ deterministic ties
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{R^\star\}$; $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{U}(R^\star)$
end while
$\mathcal{R}_1 \leftarrow \mathcal{R}$; $G \leftarrow \text{FeasibleGraph}(\mathcal{R}_1, \Theta_R)$
while G is disconnected **do**
  $\mathcal{B} \leftarrow \text{FeasibleBridges}(C_{\text{all}} \setminus \mathcal{R}, G, \Theta_R)$
  if $\mathcal{B} = \varnothing$ **then**
    return connectivity failure
  end if
  $R_b^\star \leftarrow \text{MinBridgeCost}(\mathcal{B})$ ▷ Eq. (9)
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{R_b^\star\}$; $G \leftarrow \text{UpdateFeasibleGraph}(G, R_b^\star)$
end while
return $(\mathcal{R}, G)$

3.5.1 Entry and Exit Poses

Entry and exit candidates are jointly determined by the predecessor and successor of a rectangle in the visitation sequence. If R_i is followed by R_j, a finite set of disc handoff points is sampled uniformly in their shared region or along their opposing boundaries. An entry to the first rectangle must also be reachable from q_0, and an exit from the final rectangle is selected among its remaining feasible boundary candidates. Each candidate specifies the disc center, base heading, and reference arm angle rather than only a boundary point.

For disc points on the left, right, lower, and upper boundaries, the nominal headings are π, 0, $3\pi/2$, and $\pi/2$, respectively, so that the base in its retracted configuration lies outside the corresponding rectangle boundary. This rule generates nominal candidates only. Near dense obstacles or narrow boundaries, the externally positioned base may still enter unavailable space and therefore cannot be accepted without checking.

Given a boundary pose $(\mathbf{p}_d, \theta)$ for the disc center, the base center in the retracted reference configuration is recovered as

$$\mathbf{p}_c = \mathbf{p}_d + (d_p + L)\mathbf{e}(\theta). \tag{10}$$

The state and its connection from the preceding primitive are then checked against Eqs. (3a) and (3b). If a check fails, the handoff point is moved along the same boundary, followed by trials on an adjacent boundary and with the alternative heading. Only candidates with a feasible complete connection enter the local cost comparison. If a visit admits no feasible entry–exit pair, the corresponding global sequence cannot be instantiated.

3.5.2 Starting Corner, Strip Width, and Swing Amplitude

Let ℓ_R and h_R be the rectangle dimensions parallel and perpendicular to the work strips. The local planner uses the longer axis by default to reduce lane changes, but also generates a solution along the other axis. Across the two sweep directions and their feasible starting corners, it compares the total time proxy for entry connection, coverage, lane changes, and exit connection. Thus h_R always denotes the transverse extent crossed collectively by the strips, whereas ℓ_R is the travel length of one strip.

For each sweep direction, the planner filters the four corners to retain starting points reachable from the entry and connectable to the exit; the parity of the strip count determines the coverage endpoint. It selects the feasible combination with the minimum sum of non-working motion and local coverage time. Entry, sweep direction, starting corner, strip-count parity, and exit therefore jointly determine the primitive; the start is not chosen merely as the nearest corner in Euclidean distance.

Coverage inside the rectangle follows a Boustrophedon pattern, but the work width is not fixed. It is determined by the maximum usable arm angle and disc radius. For arm limits $[\phi_{\min}, \phi_{\max}]$, the symmetric usable angle is

$$\phi_s = \min\{|\phi_{\min}|, |\phi_{\max}|, \pi/2\}. \tag{11}$$

Over one full left–right swing, the lateral width approximately reachable by the disc-center path, including disc radius, is

$$w_{\max} = 2(L \sin \phi_s + r_d). \tag{12}$$

For transverse rectangle dimension h_R, the required number of strips and actual strip width are

$$N_s = \left\lceil \frac{h_R}{w_{\max}} \right\rceil, \qquad w_s = \frac{h_R}{N_s}. \tag{13}$$

The half-amplitude required to cover a strip of width w_s is

$$\phi_w = \begin{cases} 0, & w_s \le 2r_d, \\ \arcsin\left(\dfrac{w_s/2 - r_d}{L}\right), & w_s > 2r_d, \end{cases} \tag{14}$$

clipped to $[0, \phi_s]$. This prevents systematic gaps between adjacent strips due to insufficient swing amplitude.

3.5.3 Coupling Between Base and Arm Speeds

Each strip combines straight base motion with periodic arm swinging. Let the maximum arm angular velocity be $\omega_{a,\max}$ and maximum base linear velocity $v_{\max}$. The time for the arm to move from $-\phi_w$ to $+\phi_w$ is

$$T_{\text{half}} = \frac{2\phi_w}{\omega_{a,\max}}. \tag{15}$$

The base work speed cannot be chosen from the mechanical limit alone if consecutive half-sweeps are to overlap sufficiently in the direction of travel. The method uses the following heuristic upper bound. If $\phi_w = 0$, let $v_{\text{safe}} = +\infty$; otherwise,

$$v_{\text{safe}} = \frac{2r_d}{T_{\text{half}}}. \tag{16}$$

The work speed is

$$v_{\text{work}} = \min(v_{\max}, v_{\text{safe}}). \tag{17}$$

When the arm is slow or the amplitude large, Eq. (17) reduces the base speed so that the forward distance between adjacent half-sweeps does not exceed the disc diameter. This is a conservative rule derived from local sweep spacing, not a proof of complete coverage for every continuous arm trajectory. The actual swept area is therefore still checked after local motion generation by the procedure in Section 3.7. The same relationship underlies the later experimental analysis of the arm-to-base speed ratio.

3.5.4 Time Model for First and Repeated Visits

The first visit to a rectangle includes motion to the starting corner, coverage of all strips, lane changes, turns, arm deployment and retraction, and departure from the rectangle. If strip k has length ℓ_k, lane-change segment k has length d_k, base rotation m is $\Delta\theta_m$, and arm adjustment m is $\Delta\phi_m$, the total coverage time is

$$\begin{aligned} T_{\text{cover}}(R; q^{\text{in}}, q^{\text{out}}) = &\sum_{k=1}^{N_s} \frac{\ell_k}{v_{\text{work}}} + \sum_{k} \frac{d_k}{v_{\max}} \\ &+ \sum_{m} \frac{|\Delta\theta_m|}{\omega_{\max}} + \sum_{m} \frac{|\Delta\phi_m|}{\omega_{a,\max}}. \end{aligned} \tag{18}$$

The model assumes piecewise-constant velocity and serially accumulates translation, in-place rotation, and arm adjustment. It neglects acceleration and deceleration and allows in-place turns wherever the collision constraints permit. Equation (18) is therefore a kinematic proxy for candidate ranking and nominal duration, not a time-optimal solution with actuator dynamics. Entry, exit, and starting corner affect the distances and angles and are consequently explicit arguments.

On the second or later appearance of a rectangle in the sequence, coverage has already been completed. The algorithm generates only an entry-to-exit transfer with time

$$\begin{aligned} T_{\text{transit}}(R; q^{\text{in}}, q^{\text{out}}) = &\sum_{k} \frac{d_k}{v_{\max}} + \sum_{m} \frac{|\Delta\theta_m|}{\omega_{\max}} \\ &+ \sum_{m} \frac{|\Delta\phi_m|}{\omega_{a,\max}}. \end{aligned} \tag{19}$$

A repeated visit contributes no new work coverage but has nonzero execution time. Every straight segment, in-place turn, and arm adjustment must also pass the adaptive collision check in Eq. (3a); failed entry–exit pairs are excluded from subsequent search. Allowing transit through covered rectangles makes a connected graph continuously traversable, while Eq. (19) penalizes excessive revisits in the global route cost.

3.6 Global Route Planning and Full-Trajectory Assembly

The connected graph G gives feasible transfers between rectangles but does not specify an order from the initial robot state. Let

$$\Pi = (R_{\pi_1}, R_{\pi_2}, \ldots, R_{\pi_K}) \tag{20}$$

be a visitation sequence. It must satisfy three conditions: every $R \in \mathcal{R}$ appears for a first visit at least once; every consecutive pair corresponds to an edge of G; and at least one entry pose of the first rectangle is reachable from q_0 through a collision-free initial connection. A rectangle may appear more than once so the robot can transit through a covered region to reach an unvisited branch. Before sequence generation, the algorithm attempts to connect q_0 to entry candidates of each rectangle and stores the minimum feasible initial cost as $T_{\text{start}}(q_0, R_i)$. A rectangle without such a connection cannot start the sequence.

For visit k, let q_k^{in} and q_k^{out} be the selected entry and exit states, and set $I_k = 1$ when R_{π_k} appears for the first time, otherwise $I_k = 0$. The sequence cost is

$$\begin{aligned} J(\Pi) = T_{\text{start}}(q_0, R_{\pi_1}) + \sum_{k=1}^{K} \big[&I_k T_{\text{cover}}(R_{\pi_k}; q_k^{\text{in}}, q_k^{\text{out}}) \\ &+ (1 - I_k) T_{\text{transit}}(R_{\pi_k}; q_k^{\text{in}}, q_k^{\text{out}}) \big] + \sum_{k=1}^{K-1} c_{\pi_k \pi_{k+1}}. \end{aligned} \tag{21}$$

The handoff states of adjacent edges must be consistent. Coverage cost is incurred only on a rectangle's first visit; repeated visits incur transit cost only. Equation (21) is a time proxy for comparing candidate routes, not an exact traveling-salesman objective.

Two complementary graph heuristics generate candidate sequences. The first is a multi-start randomized nearest-neighbor search. Beginning at a rectangle with a feasible initial connection, each step gathers unvisited neighbors, orders them by a weighted sum of c_{ij} and their remaining unvisited degree, and randomly chooses from the first k_{rcl} elements of this restricted candidate list. A trial is discarded if it reaches a node with no unvisited neighbor before covering all rectangles. The number of trials, k_{rcl}, and the random seed belong to Ψ_P; fixing them reproduces the experimental candidate set.

The second heuristic is an open depth-first walk on a weighted minimum spanning tree. Edge weights are c_{ij}. For each feasible root, the tree distance from the root to every node is computed, and the root-to-farthest-node path is designated as the final non-backtracking branch. All other side branches are visited in ascending edge-weight order with a return to the parent; the walk then follows the final branch to its terminal node and stops. Parent nodes and covered rectangles may therefore reappear. This provides a deterministic feasible candidate on a connected graph, but is only a feasibility fallback and is not claimed to be globally optimal in the original graph.

All candidates are first compared by Eq. (21) and then instantiated using the local primitives. Beginning with the initial connection from q_0, the algorithm creates a coverage primitive for every first visit, a transit primitive for every repeat, and a handoff motion between adjacent rectangles. A candidate is retained only if all endpoints agree and every segment passes Eqs. (3a) and (3b). The valid candidate with the smallest J is selected; its segments are shifted in time and concatenated to produce

$$q : [0, T_{\mathrm{exec}}] \to \mathcal{Q}_F. \tag{22}$$

If no candidate can be instantiated as a complete trajectory from q_0, route planning fails. Algorithm 2 summarizes the procedure.

Algorithm 2 Global route planning and trajectory assembly

Require: Rectangles $\mathcal{R}$, connected feasible graph G, initial state q_0, local primitives $\mathcal{L}$, parameters Ψ_P
Ensure: Visit sequence Π and continuous trajectory $q(t)$, or route failure

$S \leftarrow \mathrm{FeasibleStarts}(q_0, \mathcal{R}, G)$
if $S = \varnothing$ **then**
  return route failure
end if
$C \leftarrow \varnothing$
for each feasible start $s \in S$ **do**
  $\mathcal{P} \leftarrow \mathrm{RandomizedNearestNeighbor}(G, s, \Psi_P)$
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{\mathrm{OpenMstDfs}(G, s, c)\}$ ▷ farthest branch last
  for each visitation sequence $\Pi \in \mathcal{P}$ **do**
    if Π does not visit every rectangle in $\mathcal{R}$ **then**
      continue
    end if
    $a \leftarrow \mathrm{CompatibleEntryExitStates}(\Pi, G)$
    if $a \neq$ failure **then**
      $C \leftarrow C \cup \{(\Pi, a, J(\Pi, a))\}$ ▷ Eq. (21)
    end if
  end for
end for
for each (Π, a, J) in C, in increasing J **do**
  $q \leftarrow \mathrm{AssembleTrajectory}(q_0, \Pi, a, \mathcal{L})$
  Add initial and inter-rectangle connectors; cover first visits and transit repeats.
  if q is continuous and satisfies Eqs. (3a) and (3b) **then**
    return (Π, q)
  end if
end for
return route failure

3.7 Work Coverage and Full-Trajectory Coverage

Coverage is evaluated directly from the continuous trajectory rather than from a video frame rate. The trajectory is partitioned into timed segments for local coverage, turns, lane changes, arm adjustment, transit, and inter-rectangle handoff. A segment is marked as effective work only while the disc is in its working state, the base moves along a rectangular work strip, and the arm executes the prescribed swing. Entry connections, lane changes, in-place rotations, arm deployment or retraction, repeated transit visits, and inter-rectangle handoffs are marked as non-working. For any segment s, let the disc-center path be

$$\mathbf{p}_d^s(\tau), \qquad \tau \in [0, T_s]. \tag{23}$$

The disc-swept region of the segment is

$$C_s = \bigcup_{\tau \in [0,T_s]} \left\{ \mathbf{z} : \|\mathbf{z} - \mathbf{p}_d^s(\tau)\|_2 \le r_d \right\}. \tag{24}$$

Let $\mathcal{S}_{\text{work}}$ be the set of effective-work segments and $\mathcal{S}_{\text{all}}$ the set of all motion segments. The method distinguishes

$$C_{\text{work}} = \bigcup_{s \in \mathcal{S}_{\text{work}}} C_s, \qquad C_{\text{traj}} = \bigcup_{s \in \mathcal{S}_{\text{all}}} C_s. \tag{25}$$

The former contains only work-labeled disc sweeps, whereas the latter also includes disc motion during transfer, lane changes, and pose adjustment. Their respective coverage ratios are

$$\begin{aligned} \rho_{\text{work}} &= \frac{|\Pi_p(C_{\text{work}}) \cap \mathcal{P}_F|}{|\mathcal{P}_F|}, \\ \rho_{\text{traj}} &= \frac{|\Pi_p(C_{\text{traj}}) \cap \mathcal{P}_F|}{|\mathcal{P}_F|}, \end{aligned} \tag{26}$$

where $\Pi_p(\cdot)$ rasterizes a continuous world-coordinate region into a pixel set. The primary coverage measure is ρ_{work} because it corresponds to explicitly labeled effective-work motion. The value ρ_{traj} describes how much free area is swept over the complete motion. If the disc remains in contact with the floor during non-working transfers, it captures additional physical sweeping; if the disc is lifted, it is only an auxiliary measure of path reach or potential sweep and must not be conflated with effective work coverage.

To avoid dependence on an arbitrary sampling frame rate, $\mathbf{p}_d^s(\tau)$ is recursively subdivided. A time interval $[\tau_a, \tau_b]$ is accepted only when the endpoint chord length is no greater than η_ℓ and the deviation of the midpoint from that chord is no greater than η_c. Both are chosen no larger than $s_p/2$, bounding discretization error by map resolution. Let the final subdivision points be $\mathbf{c}_0, \ldots, \mathbf{c}_m$. Rasterizing the radius-r_d capsule between every adjacent pair yields

$$\widehat{C}_s = \Pi_p\left(\bigcup_{j=0}^{m-1} \text{Capsule}(\mathbf{c}_j, \mathbf{c}_{j+1}, r_d) \right). \tag{27}$$

The capsule union is constructed in world coordinates before conversion by Π_p, avoiding a type mismatch between continuous regions and pixel sets. This method is more stable than drawing discs only at fixed-rate samples, especially during fast arm motion.

Finally, nominal execution time and planning time are

$$T_{\text{exec}} = \sum_{s \in \mathcal{S}_{\text{all}}} T_s, \qquad T_{\text{plan}} = t_{\text{eval,end}} - t_{\text{raster,start}}. \tag{28}$$

The first follows the piecewise-constant and serial-timing assumptions used for the local primitives. The second is measured as wall-clock time in one process, from conservative-grid generation through continuous-sweep evaluation. It includes decomposition, connectivity completion, route search, trajectory assembly, and coverage computation, but excludes map-file reading, output writing, and visualization. Thus ρ_{work}, ρ_{traj}, T_{exec}, and T_{plan} match the output definitions in Fig. 3.

4 Experiments and Results

Three groups of experiments evaluate the proposed swing-arm CPP method. The first compares it with Fields2Cover to clarify the difference between active arm sweeping and fixed-width strip-based CPP. The second compares rectangular-candidate scan strategies to determine how decomposition affects coverage quality and redundancy in complex spaces. The third varies base speed at a fixed arm angular velocity to expose how velocity matching affects coverage and execution time.

All three groups use two base maps. `map_test1` contains four regular rectangular obstacles and tests basic obstacle avoidance and coverage. `parkingGraph` contains densely distributed small obstacles and better resembles a construction floor partitioned by columns, openings, and local structures. The scan-strategy experiment additionally uses three directional maps:

- `scan_comb_corridors`: horizontal comb-like corridors;
- `scan_mixed_weave`: interwoven horizontal and vertical structures; and
- `scan_offset_rooms`: rooms connected with offsets.

These structures distinguish the effects of row-only, column-only, and bidirectional scanning.

Every map is a binary occupancy image in which white pixels are free target area and black pixels are obstacles or unavailable regions. The resolution is 0.5 m/pixel. Except in the speed experiment, the same parameters are used throughout: disc radius $r_d = 0.75$ m, arm length $L = 1.20$ m, base width 0.80 m, maximum base linear velocity 2.00 m/s, maximum base angular velocity 30°/s, and maximum arm angular velocity 90°/s. The minimum rectangle side length is 2.5 m, the greedy-selection weights are $\alpha = 0.5$ and $\beta = 2.0$, and the random seed is 42.

4.1 Comparison with Fields2Cover

Fields2Cover is an appropriate fixed-width baseline and a representative open-source framework for this class of problem. It is supported by a peer-reviewed publication and continuing open-source development, integrates multiple coverage-planning algorithms and objectives, and includes headland generation, swath generation, route ordering, and curvature-constrained path connections. It supports non-convex regions, internal obstacles, fixed working and vehicle widths, turning radii, and speed parameters. Its functional completeness, algorithmic integration, and reproducibility make it a strong contemporary baseline for fixed-width CPP [18, 19]. The comparison is not intended to rank implementation details, but to expose differences in coverage geometry and executability between an active swing-arm model and an integrated, fixed-width strip-planning framework in obstacle-rich spaces.

For Fields2Cover, the working width is 1.50 m, equal to the diameter of the proposed method's 0.75 m-radius disc. With base width 0.80 m and safety margin 0.05 m, the original free region is eroded by 0.45 m. Both methods are evaluated over their complete trajectories. For the proposed method, coverage is the area swept by the disc along the base–arm trajectory; for Fields2Cover, it is the fixed-width tool sweep along work, turning, and transfer segments. To align evaluation with the actual planning domain, the Fields2Cover denominator is the eroded free-space area. Its fixed-width sweep uses flat strip ends and mitered path corners. The work-strip speed is 2.0 m/s, while turns and connecting paths use 0.5 m/s.

Table 2: Comparison between the proposed method and Fields2Cover.

Map	Method	Evaluation area (m^2)	Covered area (m^2)	Coverage ratio (%)	Execution/planning time (s/s)	Path check
`map_test1`	Proposed	611.00	575.00	94.11	854.73/1.298	Passed
	Fields2Cover	533.17	418.00	78.40	295.50/0.075	4.51 m^2 outside
`parkingGraph`	Proposed	4294.00	4110.50	95.73	5844.73/17.981	Passed
	Fields2Cover	4046.79	3756.85	92.84	2883.81/6.339	20.00 m^2 outside

The evaluated areas in Table 2 differ between methods, so their coverage ratios should not be interpreted as same-domain percentage-point differences. The Fields2Cover routes fail the path check; their execution

and planning times are reported only for the generated outputs.

On `map_test1`, Fields2Cover has shorter generation and nominal execution times with a 1.50 m fixed width equal to the disc diameter. Its complete trajectory sweeps 418.00 m^2 (78.40%), compared with 575.00 m^2 (94.11%) for the proposed method. Its connecting segments, however, sweep 4.51 m^2 outside the admissible region. The recorded 295.50 s execution time and 0.075 s planning time therefore describe an invalid generated path rather than an executable solution. This illustrates the principal benefit of the swing-arm configuration: the base need not drive along every strip that the disc must cover. Arm motion extends the covered region and creates a wider time-varying sweep within the same free space.

On `parkingGraph`, the proposed method covers 4110.50 m^2, or 95.73% of its evaluation domain. Fields2Cover sweeps 3756.85 m^2 (92.84%) along the complete trajectory. Yet its connecting path for the 1.50 m tool leaves the free space, creating 20.00 m^2 of invalid base sweep; the corresponding 2883.81 s execution time and 6.339 s planning time are likewise not valid baseline scores. The problem is concentrated near dense local obstacles. Although individual swaths fit inside the eroded region, curvature-constrained connections between swaths require additional room. Without a broad transfer area analogous to an agricultural headland, these connections can cross the boundary of the original free space. Active arm sweeping partially decouples base travel from end-disc coverage and preserves greater coverage capability and path continuity in such complex regions.

4.2 Effect of the Rectangle-Candidate Scan Strategy

The preceding comparison shows that an active arm can create coverage patterns unlike those of fixed-width strips. Realizing this advantage consistently also requires a decomposition that represents free space appropriately. A candidate generator biased toward one scan direction can produce fragmented or heavily overlapping rectangles; even when the resulting trajectory is executable, unnecessary transfers may follow. This experiment traces how scan direction propagates through the rectangle set, visitation sequence, and complete trajectory.

The compared strategies are row-only, column-only, and row-then-column scanning. Robot parameters, minimum rectangle size, greedy weights, random seed, connectivity completion, visitation ordering, and local coverage generation are fixed. Measures include greedy coverage, final decomposition coverage, decomposition overlap, full-trajectory coverage, and total execution time, distinguishing representation quality from execution performance. Candidates are first generated and greedily selected from a binary map. After hole filling and connectivity completion, the same trajectory generation and disc-sweep evaluation method is applied in every case.

Table 3: Results of three candidate-scan strategies on five maps.

Map	Strategy	Greedy coverage	Final partition coverage	Partition overlap	Trajectory coverage	Total time (s)
`map_test1`	row-only	72.01%	100.00%	34.21%	93.04%	696.89
	col-only	72.01%	100.00%	34.21%	93.04%	696.89
	row-then-col	100.00%	100.00%	0.00%	94.11%	854.73
`parkingGraph`	row-only	87.32%	100.00%	75.34%	95.73%	5844.73
	col-only	87.32%	100.00%	75.34%	95.90%	5896.03
	row-then-col	87.32%	100.00%	75.34%	95.73%	5844.73
Comb corridors	row-only	87.18%	100.00%	34.46%	93.82%	2065.09
	col-only	86.53%	100.00%	31.46%	93.94%	2032.55

Map	Strategy	Greedy coverage	Final partition coverage	Partition overlap	Trajectory coverage	Total time (s)
	row-then-col	96.22%	100.00%	27.13%	94.01%	1986.42
Mixed weave	row-only	84.23%	100.00%	47.89%	95.26%	1532.65
	col-only	83.17%	100.00%	52.59%	95.49%	1575.63
	row-then-col	96.76%	100.00%	40.74%	95.70%	1522.77
Offset rooms	row-only	82.95%	100.00%	38.78%	91.97%	1496.73
	col-only	81.81%	100.00%	36.74%	91.51%	1443.31
	row-then-col	99.14%	100.00%	24.46%	92.06%	1374.38
Five-map mean	row-only	–	–	–	93.96%	2327.22
	col-only	–	–	–	93.98%	2328.88
	row-then-col	–	–	–	94.32%	2316.60

The two general maps first establish basic behavior in non-directional scenes. On `map_test1`, row-then-column achieves the highest full-trajectory coverage (94.11%) and reduces partition overlap to zero, but its 854.73 s execution time exceeds the 696.89 s of either one-direction strategy. In this regular obstacle scene, bidirectional scanning trades longer execution for higher coverage. On `parkingGraph`, row-then-column and row-only select the same rectangles and give identical results. Column-only provides slightly higher coverage (95.90%), whereas row-only and row-then-column tie for the shortest total time (5844.73 s). Bidirectional scanning therefore does not dominate every metric in every scene. When free space has a pronounced principal direction or row candidates already capture the structure, a one-direction strategy may be faster or marginally improve coverage.

The three directional maps then isolate scan-direction bias; they are not intended to prove that bidirectional scanning is optimal on arbitrary maps. Horizontal corridors, orthogonal interweaving, and offset connections amplify residual regions created by a single scan direction. On all three maps, row-then-column gives the highest greedy coverage and lowest partition overlap. Column candidates supplement regions that row candidates represent poorly rather than merely increasing the rectangle count. This improvement propagates to execution: row-then-column gives both the highest full-trajectory coverage and shortest time on all three maps, respectively 94.01%/1986.42 s, 95.70%/1522.77 s, and 92.06%/1374.38 s. Their mean is 93.93%/1627.86 s. When a map truly contains residual structures in both directions, complementary row and column candidates reduce directional bias and redundant visitation while increasing useful sweep along the complete trajectory.

Across all five maps, row-then-column averages 94.32% coverage, compared with 93.96% for row-only and 93.98% for column-only. Its mean execution time of 2316.60 s is also slightly below 2327.22 s and 2328.88 s. It is therefore best characterized as a robust default: it is more likely to preserve high coverage without a substantial time penalty when horizontal and vertical residual structures coexist. For a regular region with a single principal direction, a one-direction strategy may still be preferable under the specific coverage–schedule trade-off. Figure 8 visualizes the directional complementarity on `scan_mixed_weave`.

4.3 Effect of Base–Arm Velocity Coupling on Coverage

The first two experiments address route and decomposition quality. Swing-arm coverage also has an execution issue that is less prominent in fixed-width CPP: the same geometric route produces different disc-sweep densities at different base speeds. As the base advances faster, it travels farther between adjacent arm sweeps; slower travel improves coverage but substantially increases task time. Reporting

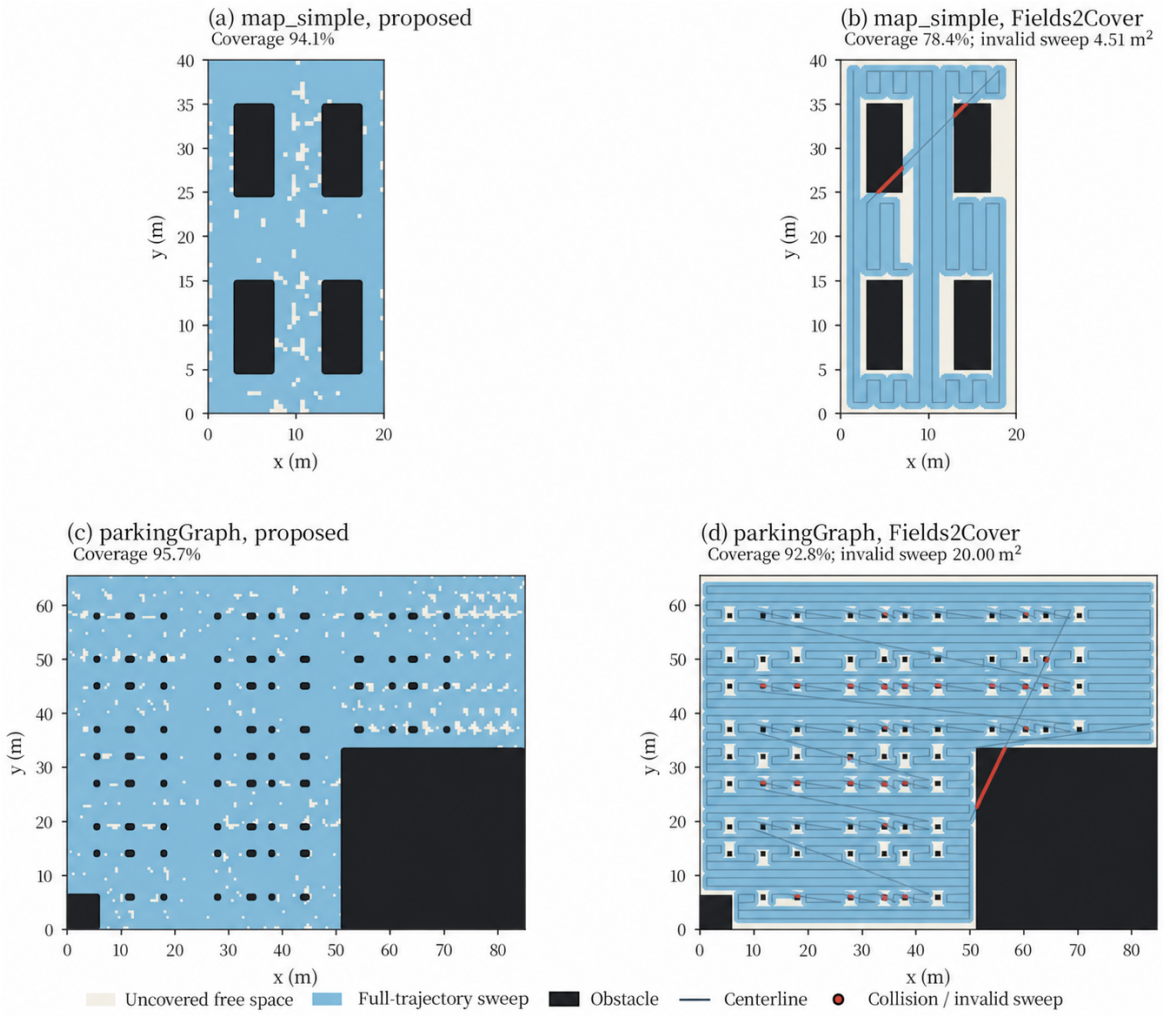


Figure 7: Coverage paths and swept regions for the proposed method and Fields2Cover. Panels (a) and (b) show `map_test1` (labeled `map_simple` inside the image), and panels (c) and (d) show `parkingGraph`; the proposed method is on the left and Fields2Cover on the right. Blue is the full-trajectory swept region, white or light areas are uncovered free space in the evaluation domain, black is obstacle or unavailable space, and red marks base collisions or invalid sweep. The Fields2Cover route fails the base-collision check on both maps: 4.51 m^2 lies outside the valid region on `map_test1` and 20.00 m^2 on `parkingGraph`. Its displayed coverage and timing values therefore describe generated, non-executable routes.

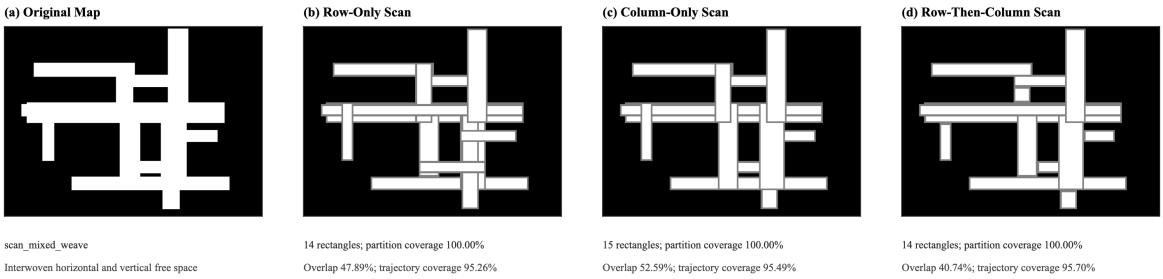


Figure 8: Candidate-scan strategies on `scan_mixed_weave`: (a) original map; (b) row-only decomposition; (c) column-only decomposition; and (d) row-then-column decomposition. The map contains narrow horizontal and vertical free regions. Single-direction candidate sets leave residual structures in the other orientation, whereas sequential row and column candidates form a more complete decomposition with less overlap.

a geometric path without analyzing its velocity relationship to the arm therefore cannot establish the coverage quality and time cost of actual execution.

This experiment tests the analysis in Section 3.5.3. At fixed arm angular velocity, increasing base speed should shorten execution but enlarge the spacing between adjacent sweeps and reduce coverage. Keeping the rectangle set, visitation order, and local-work structure fixed isolates the effect of base speed.

The arm angular velocity is fixed at $\omega_a = 90°/s$. The same planned solutions are executed on `map_test1` and `parkingGraph` with base speed increasing from 0.5 to 3.0 m/s. A velocity ratio places linear and angular speed on a common scale:

$$\rho = \frac{v}{\omega_a},$$

with units of m/degree. It is the base distance traveled while the arm rotates through $1°$. A larger ρ increases forward spacing between adjacent sweeps and makes coverage gaps more likely. Coverage, total time, and time per unit covered area are recorded to expose the quality–efficiency trade-off.

Table 4: Full-trajectory coverage and execution time at different base speeds.

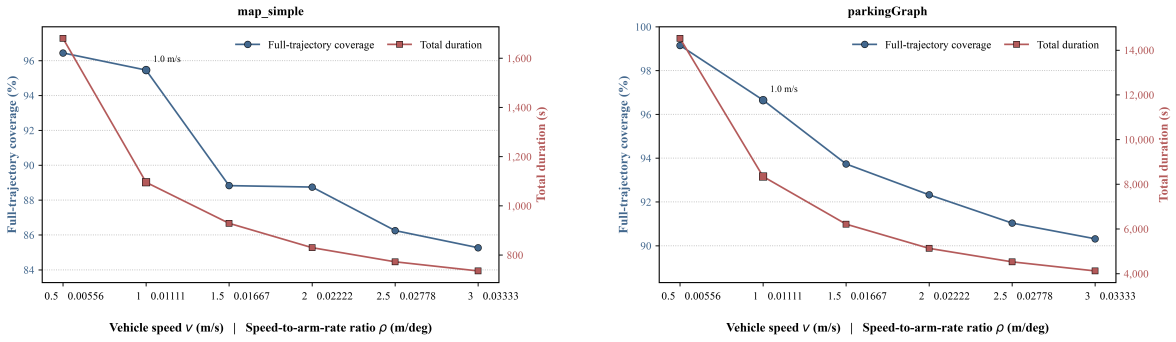
Map	Base speed (m/s)	ρ (m/degree)	Full-path coverage	Total time (s)	Time per covered area (s/m^2)
`map_test1`	0.5	0.00556	96.44%	1680.45	2.852
	1.0	0.01111	95.46%	1095.53	1.878
	1.5	0.01667	88.83%	928.81	1.711
	2.0	0.02222	88.75%	829.74	1.530
	2.5	0.02778	86.25%	772.26	1.465
	3.0	0.03333	85.27%	735.12	1.411
`parkingGraph`	0.5	0.00556	99.15%	14525.14	3.412
	1.0	0.01111	96.65%	8346.21	2.011
	1.5	0.01667	93.74%	6214.22	1.544
	2.0	0.02222	92.33%	5130.64	1.294
	2.5	0.02778	91.03%	4526.86	1.158
	3.0	0.03333	90.32%	4120.11	1.062

Table 4 shows a clear coverage–time trade-off, although the full-trajectory results make it more moderate than an analysis of work strips alone might suggest. Total time decreases monotonically with base speed: from 1680.45 to 735.12 s (56.25%) on `map_test1` and from 14525.14 to 4120.11 s (71.63%) on `parkingGraph`. As ρ increases from 0.00556 to 0.03333 m/degree, coverage decreases from 96.44% to 85.27% and from 99.15% to 90.32%, reductions of 11.17 and 8.83 percentage points. Larger sweep spacing therefore weakens coverage continuity, but no tested condition makes the full-trajectory coverage unusably low.

Time per unit covered area further clarifies the trade-off. It decreases continuously with base speed on both maps: from 2.852 to 1.411 s/m^2 on `map_test1` and from 3.412 to 1.062 s/m^2 on `parkingGraph`, improvements of 50.53% and 68.88%. A speed of 3.0 m/s gives both the shortest total time and lowest time per covered area, at the cost of coverage ratios of 85.27% and 90.32%. Conversely, 0.5 m/s gives the highest coverage on both maps. If both maps must retain at least 95% full-trajectory coverage, 1.0 m/s is the highest tested admissible speed, while already reducing total time by 34.81% and 42.54% relative to 0.5 m/s. Base speed should therefore be selected jointly with arm angular velocity according to a minimum-coverage constraint and schedule objective, rather than treated as an incidental parameter after geometric planning. Smaller ρ favors coverage quality; larger ρ favors time efficiency when a coverage loss of roughly 9–11 percentage points is acceptable.

4.4 Discussion

The three experiments support one central conclusion: the contribution is not simply another decomposition, but the use of coordinated base motion and end-disc swinging throughout map decomposition, trajectory



(a) `map_test1` (b) `parkingGraph`

Figure 9: Effect of the base-to-arm velocity ratio on full-trajectory coverage and execution time. The horizontal axis is $\rho = v/\omega_a$; the secondary top axis gives the corresponding base speed. Blue denotes full-trajectory coverage and red total execution time. Both curves use the data in Table 4.

generation, and swept-area evaluation, so that coverage is determined by the disc's actual continuous motion. The Fields2Cover comparison shows that fixed-width strips are efficient in simple scenes, but coverage capability and connection feasibility can be constrained in dense obstacle fields; the time-varying end-disc sweep enlarges the effective workspace. The scan experiment shows that decomposition must reflect map structure and that bidirectional candidates reduce the representation bias of a single scan direction. The speed experiment shows that swing-arm CPP must jointly consider route geometry and execution speed, because matching base speed to arm angular velocity directly determines coverage quality.

These findings do not imply unconditional superiority over a fixed-width platform. In regular open areas, a strip-based method can obtain acceptable coverage with a shorter path and lower time. The value of the swing arm is greatest at boundaries a fixed base cannot closely follow, in obstacle-dense regions, and on wet concrete where fewer base maneuvers are desirable. Platform and parameter selection can therefore depend on site structure: large open regions favor efficient fixed-width sweeps, while complex corners and multi-obstacle regions favor arm-assisted coverage. The two could also operate in separate zones of the same floor.

The method has several limitations. First, rectangle candidates and visitation sequences are generated heuristically. They provide feasible and interpretable routes but no global optimality guarantee; many narrow corridors can still increase the rectangle count and repeated transit. Second, the input is a static two-dimensional map and does not explicitly account for workers, material stacks, localization drift, or changes in concrete condition during construction. Third, the speed analysis focuses on coverage density under base linear and arm angular velocities; it does not yet model full chassis dynamics, disc pressure, surface-quality feedback, or control error. Overall, the experiments identify explicit modeling of the end-tool sweep, area decomposition, and velocity coupling as the features that distinguish swing-arm troweling CPP from conventional fixed-width planning and define the next steps toward field deployment.

5 Conclusion and Future Work

Obstacles and narrow connections in complex construction maps prevent a swing-arm troweling robot from obtaining a continuous, executable coverage route from either a fixed tool footprint or a static reachability region alone. This paper presented a hierarchical CPP method for known two-dimensional environments with obstacles. Bidirectional rectangular candidates, hole filling, and connectivity completion represent the workable area; a feasible graph organizes global visitation; and first visits and repeated transit visits are instantiated as coordinated base–arm work motions and transfer motions. Planning and evaluation preserve the geometry of the base, arm, and circular disc, and coverage and nominal execution time are calculated from the disc's actual sweep along the continuous trajectory. The method thus connects map decomposition, region visitation, and end-tool coverage verification in one complete pipeline.

On `map_test1` and `parkingGraph`, the proposed method achieves full-trajectory coverage ratios of 94.11% and 95.73%, respectively, and both routes pass the path check. Although the fixed-width Fields2Cover baseline generates shorter paths, its connecting segments leave the valid region by 4.51 m^2 and 20.00 m^2 on the two maps, so the associated nominal times cannot be treated as executable-route results. The comparison demonstrates the practical value of adjustable relative motion between base travel and disc operation in dense obstacle fields. In the five-map scan-strategy study, row-then-column candidates give the highest mean full-trajectory coverage (94.32%) and the lowest mean execution time (2316.60 s), indicating that complementary row and column directions reduce representation bias; a single-direction strategy can nevertheless be preferable in some regular scenes. The speed study further shows that increasing base speed relative to arm angular velocity enlarges the spacing between sweeps and creates a coverage–time trade-off. Route geometry and execution parameters must therefore be selected together.

The current method relies on heuristic decomposition and visitation ordering and does not guarantee global optimality. It assumes a known static two-dimensional map, and its evaluation does not yet include complete chassis dynamics, localization error, disc pressure, or surface-quality feedback. Future work will jointly optimize decomposition and visitation, introduce online perception and local replanning, and validate trajectory execution and finishing quality on a physical swing-arm troweling robot. The framework may also serve as a decision-support tool for construction-map partitioning and base–arm motion configuration.

References

[1] U.S. Bureau of Labor Statistics. Census of fatal occupational injuries summary, 2024, 2026. URL: https://www.bls.gov/news.release/cfoi.nr0.htm.

[2] National Institute for Occupational Safety and Health. Silica and worker health, 2024. URL: https://www.cdc.gov/niosh/silica/about/index.html.

[3] T. S. Abdelhamid and J. G. Everett. Physiological demands of concrete slab placing and finishing work. *Journal of Construction Engineering and Management*, 125(1):47–52, 1999. doi:10.1061/(ASCE)0733-9364(1999)125:1(47).

[4] J. Albers, S. Russell, and K. Stewart. Concrete leveling techniques—a comparative ergonomic assessment. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, volume 48, pages 1357–1361, 2004. doi:10.1177/154193120404801217.

[5] M. Kobayashi, M. Soma, and T. Shigihara. Development and implementation of troweling robot for concrete floor finishing. In *Proceedings of the 13th International Symposium on Automation and Robotics in Construction*, pages 973–980, 1996. doi:10.22260/ISARC1996/0113.

[6] Methvin. Concrete finishing: Estimating man-hour norms and planning production rates, 2026. Accessed 2026-08-27. URL: https://methvin.co/production-rates/concrete-work/concrete-finishes/.

[7] M. Hwangbo, B.-J. You, S.-R. Oh, K. Y. Yi, S. H. Kim, S. Y. Chung, S. S. Han, I. H. Suh, and W.-K. Chung. Development of an unmanned autonomous concrete floor robotic troweling system. In *Proceedings of the 16th International Symposium on Automation and Robotics in Construction*, pages 79–84, 1999. doi:10.22260/ISARC1999/0012.

[8] J.-R. Lin, S. Zhou, P. Pan, R. Cai, and G. Chen. Sataoi: Delimitating area of interest for swing-arm troweling robot for construction. In *Proceedings of the 42nd International Symposium on Automation and Robotics in Construction*, pages 389–396, 2025. URL: https://www.iaarc.org/publications/2025_proceedings_of_the_42nd_isarc_montreal_canada/sataoi_delimitating_area_of_interest_for_swing-arm_troweling_robot_for_construction.html, doi:10.22260/ISARC2025/0052.

[9] E. Galceran and M. Carreras. A survey on coverage path planning for robotics. *Robotics and Autonomous Systems*, 61(12):1258–1276, 2013. doi:10.1016/j.robot.2013.09.004.

[10] T.-K. Lee, S.-H. Baek, Y.-H. Choi, and S.-Y. Oh. Smooth coverage path planning and control of mobile robots based on high-resolution grid map representation. *Robotics and Autonomous Systems*, 59(10):801–812, 2011. doi:10.1016/j.robot.2011.06.002.

[11] J. Storms, K. Chen, and D. Tilbury. Modeling teleoperated robot driving performance as a function of environment difficulty. *IFAC-PapersOnLine*, 49(32):216–221, 2016. doi:10.1016/j.ifacol.2016.12.217.

[12] Ontario Construction Secretariat. 2024 contractor survey, 2024. URL: https://iciconstruction.com/wp-content/uploads/2024/03/OCS_C-SURVEY_RPT2024-FINAL-E-Version.pdf.

[13] J. M. Davila Delgado, L. Oyedele, A. Ajayi, L. Akanbi, O. Akinade, M. Bilal, and H. Owolabi. Robotics and automated systems in construction: Understanding industry-specific challenges for adoption. *Journal of Building Engineering*, 26:100868, 2019. doi:10.1016/j.jobe.2019.100868.

[14] H. Choset and P. Pignon. Coverage path planning: The boustrophedon cellular decomposition. In *Proceedings of the 1st International Conference on Field and Service Robotics*, pages 203–209, 1998.

[15] Y. Gabriely and E. Rimon. Spanning-tree based coverage of continuous areas by a mobile robot. *Annals of Mathematics and Artificial Intelligence*, 31:77–98, 2001. doi:10.1023/A:1016610507833.

[16] R. Bähnemann, N. Lawrance, J. J. Chung, M. Pantic, R. Siegwart, and J. Nieto. Revisiting boustrophedon coverage path planning as a generalized traveling salesman problem. In *Field and Service Robotics*, pages 277–290, 2021. doi:10.1007/978-981-15-9460-1_20.

[17] Autonomous Systems Lab, ETH Zurich. polygon_coverage_planning: Coverage planning in general polygons with holes, 2026. Accessed 2026-06-28. URL: https://github.com/ethz-asl/polygon_coverage_planning.

[18] G. Mier, J. Valente, and S. de Bruin. Fields2cover: An open-source coverage path planning library for unmanned agricultural vehicles. *IEEE Robotics and Automation Letters*, 8(4):2166–2172, 2023. doi:10.1109/LRA.2023.3248439.

[19] Fields2Cover contributors. Fields2cover: Robust and efficient coverage paths for autonomous agricultural vehicles, 2026. Accessed 2026-06-28. URL: https://github.com/Fields2Cover/Fields2Cover.

[20] Nobleo Technology. Full coverage path planner, 2026. Accessed 2026-06-28. URL: https://github.com/nobleo/full_coverage_path_planner.

[21] S. Thakar, R. K. Malhan, P. M. Bhatt, and S. K. Gupta. Area-coverage planning for spray-based surface disinfection with a mobile manipulator. *Robotics and Autonomous Systems*, 147:103920, 2022. doi:10.1016/j.robot.2021.103920.

[22] D. H. Shin and D. H. Han. Open-loop velocity control of concrete floor finishing robots. *Journal of Intelligent & Robotic Systems*, 36(3):285–300, 2003. doi:10.1023/A:1023037004444.